Software Testing

ISTQB / ISEB Foundation Exam Practice

Testing throughout the Software Life Cycle

1 Principles	2 Lifecycle	3 Static testing
4 Test design techniques	5 Management	6 Tools

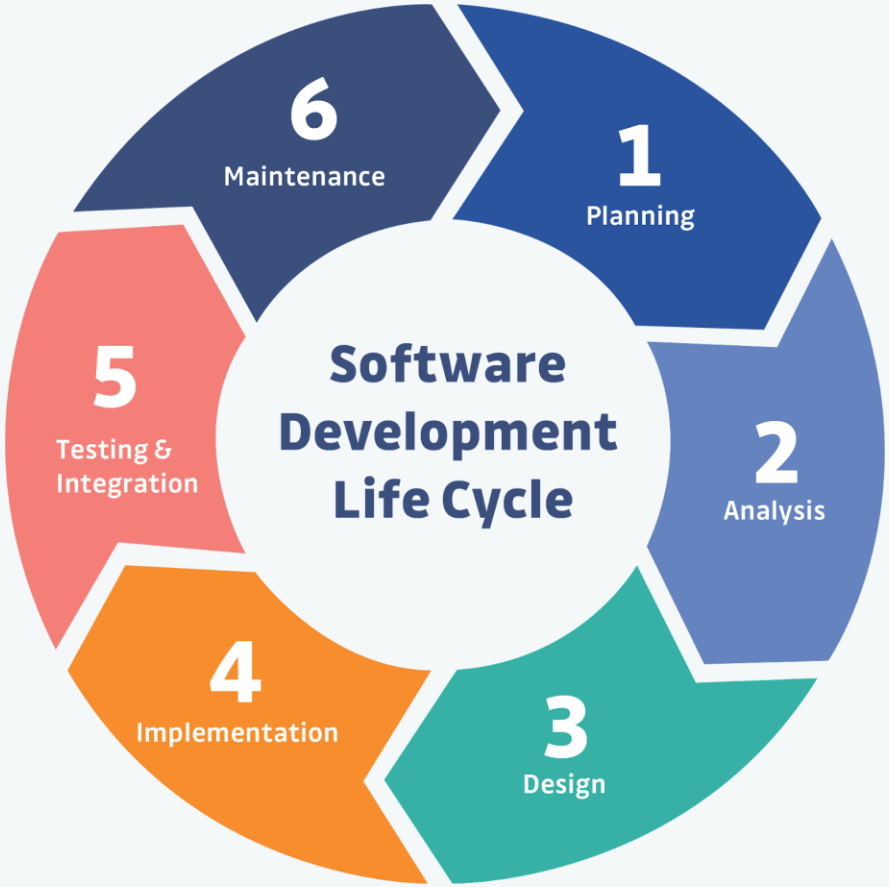
Chapter 2

CONTENT

- Software Development Life Cycle Models
- Test levels
- Test types
- Maintenance testing

Software Development Lifecycle

A software development lifecycle model describes the types of activity performed at each stage in a software development project, and how the activities relate to one another logically and chronologically.



Characteristics of Good Testing
[in any software lifecycle model]

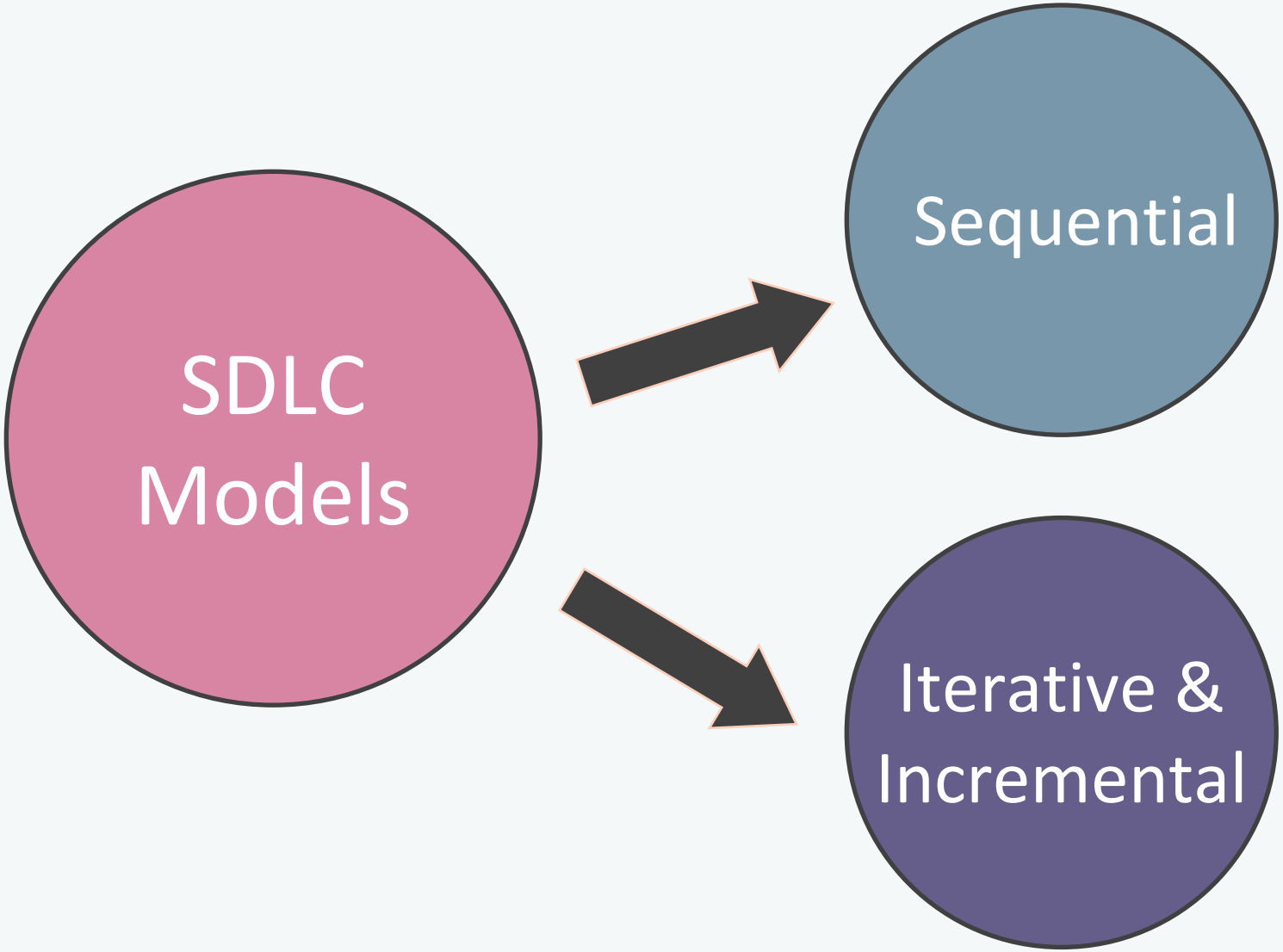
For every development activity, there is a corresponding test activity.

Each test level has test objectives specific to that level.

The analysis & design of tests for a given test level should begin during the corresponding software development activity.

Tester participate in discussion to help define & and refine requirements and design and are involved in reviewing work products.

Software Development Lifecycle Models

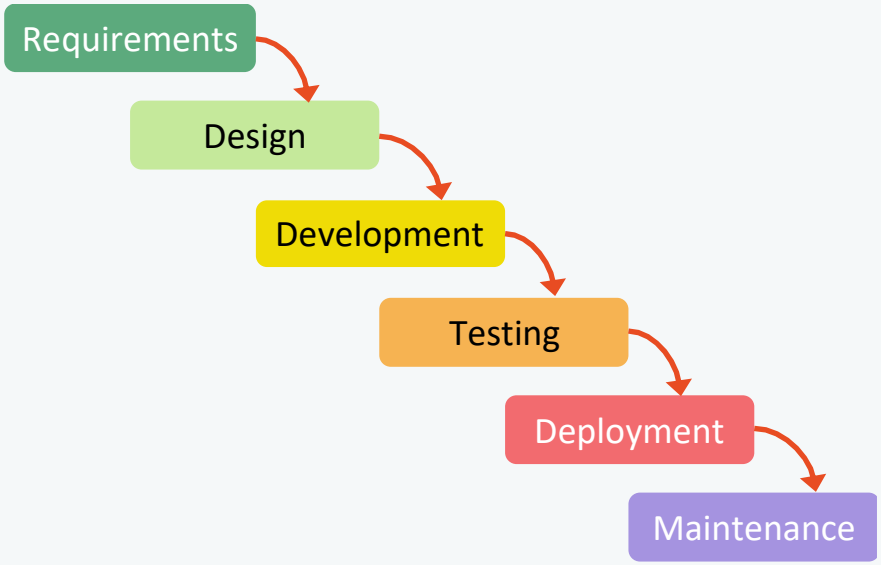


Sequential Development Models

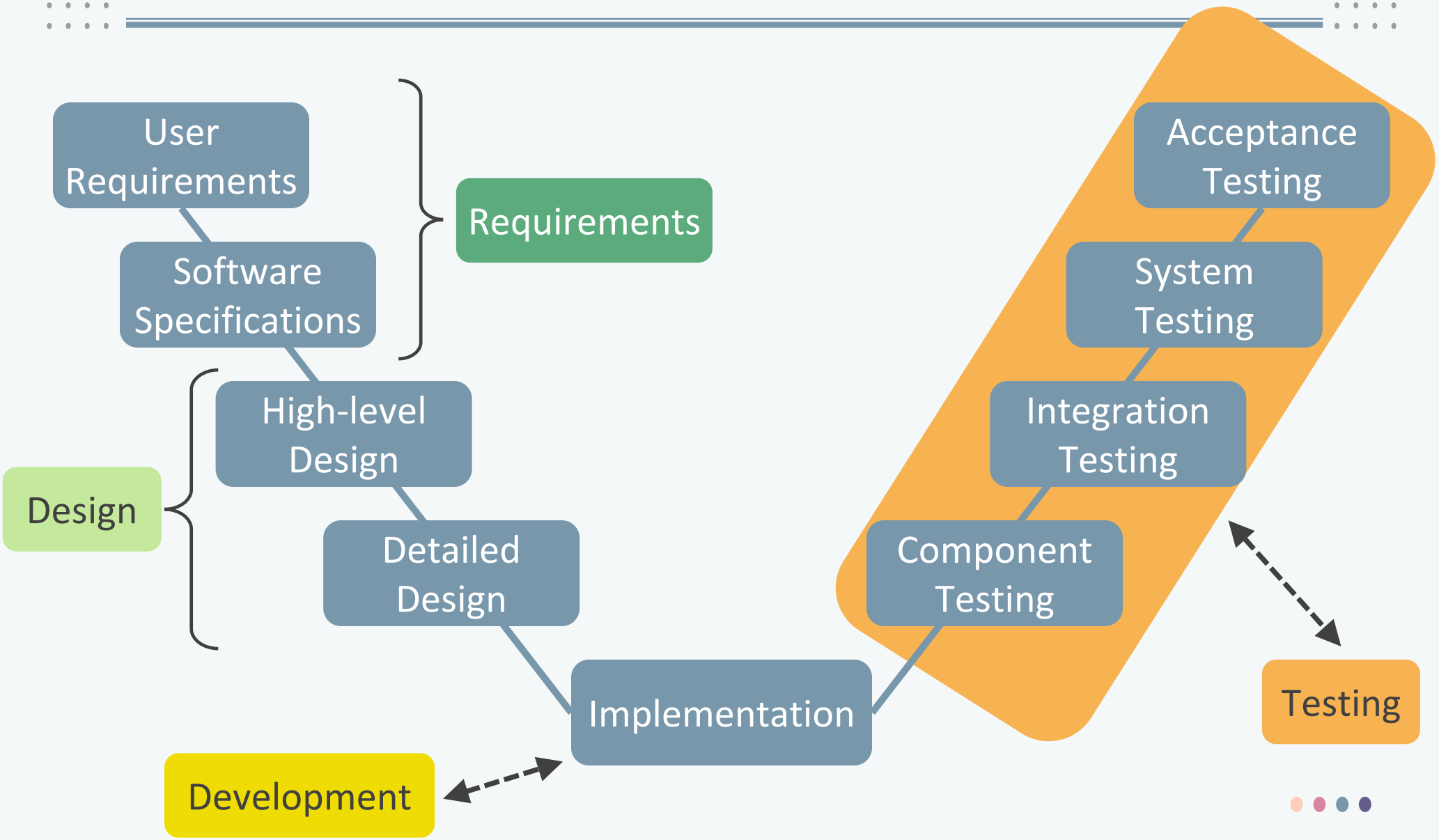
- A sequential development model describes the software development process as a linear, sequential flow of activities.
- Any phase in the development process should begin when the previous phase is complete.
- In theory, there is no overlap of phases, but in practice, it is beneficial to have early feedback from the following phase

Waterfall Model

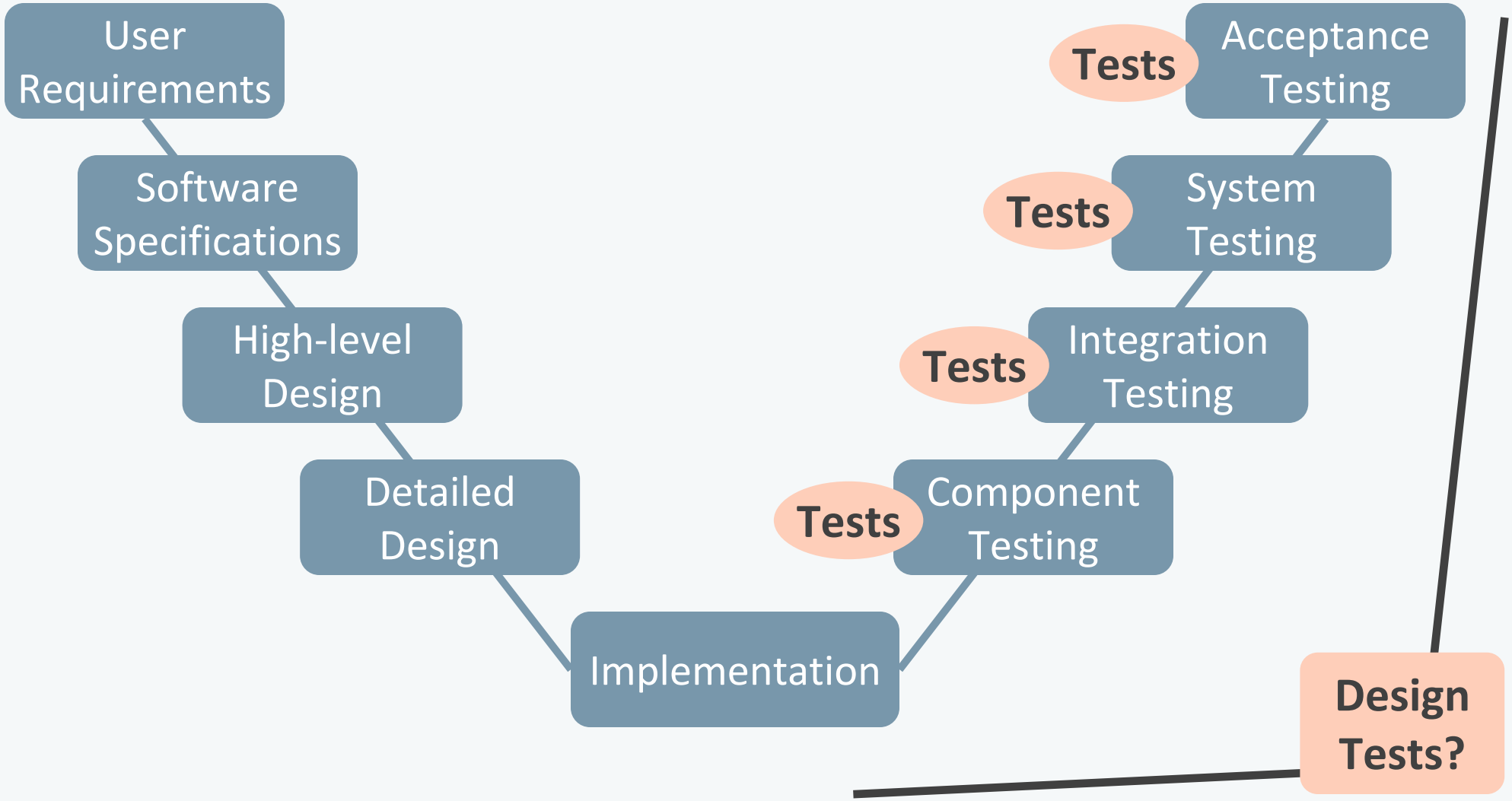
- The development activities are completed one after another.
- Testing tends to happen towards the end of the life cycle → defects are detected close to the live deployment date.
- It is difficult to get feedback passed backwards up the waterfall & cost of change is high.



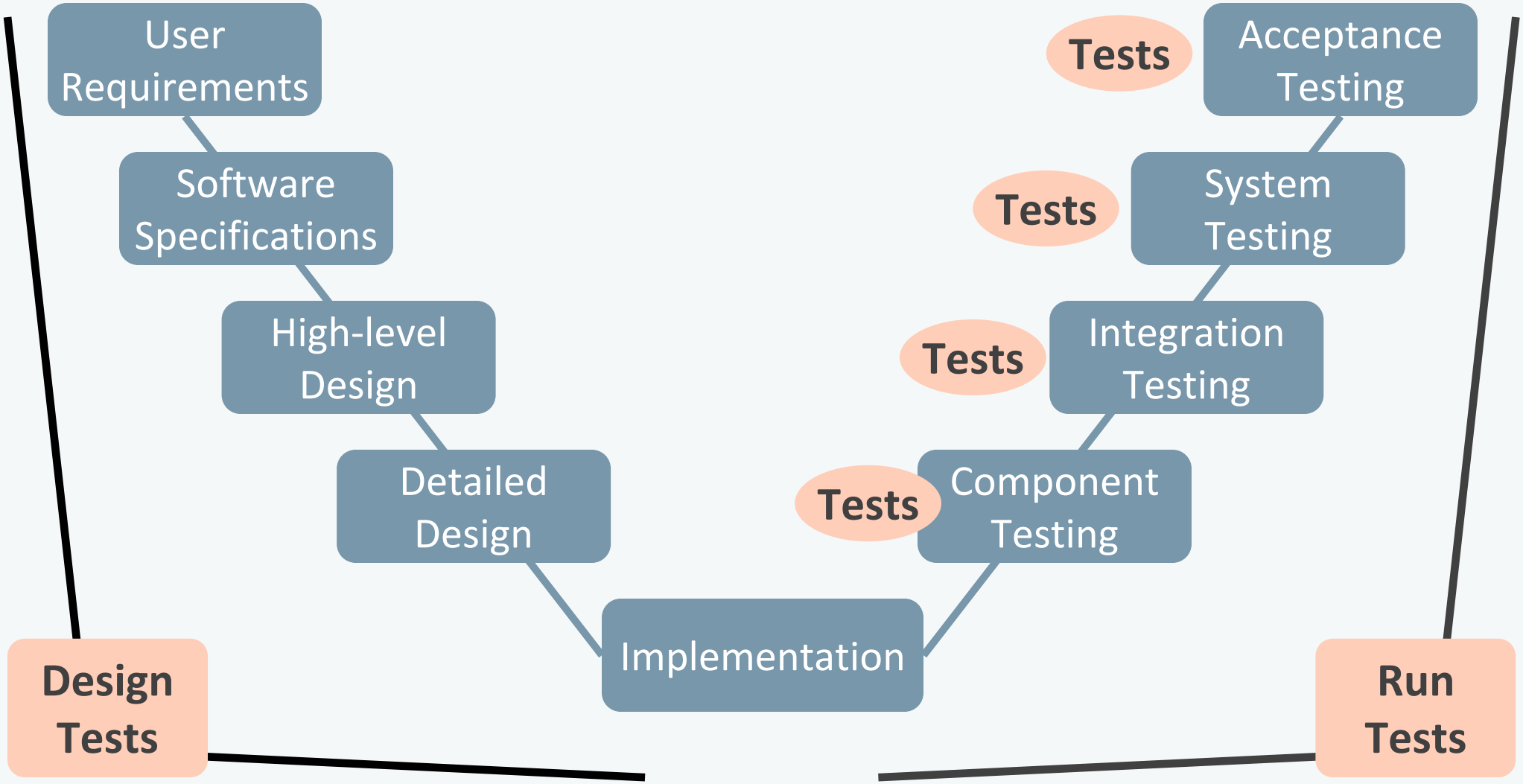
V-Model: Test Levels



V-Model: Late Test Design



V-Model: Early Test Design



Early test design

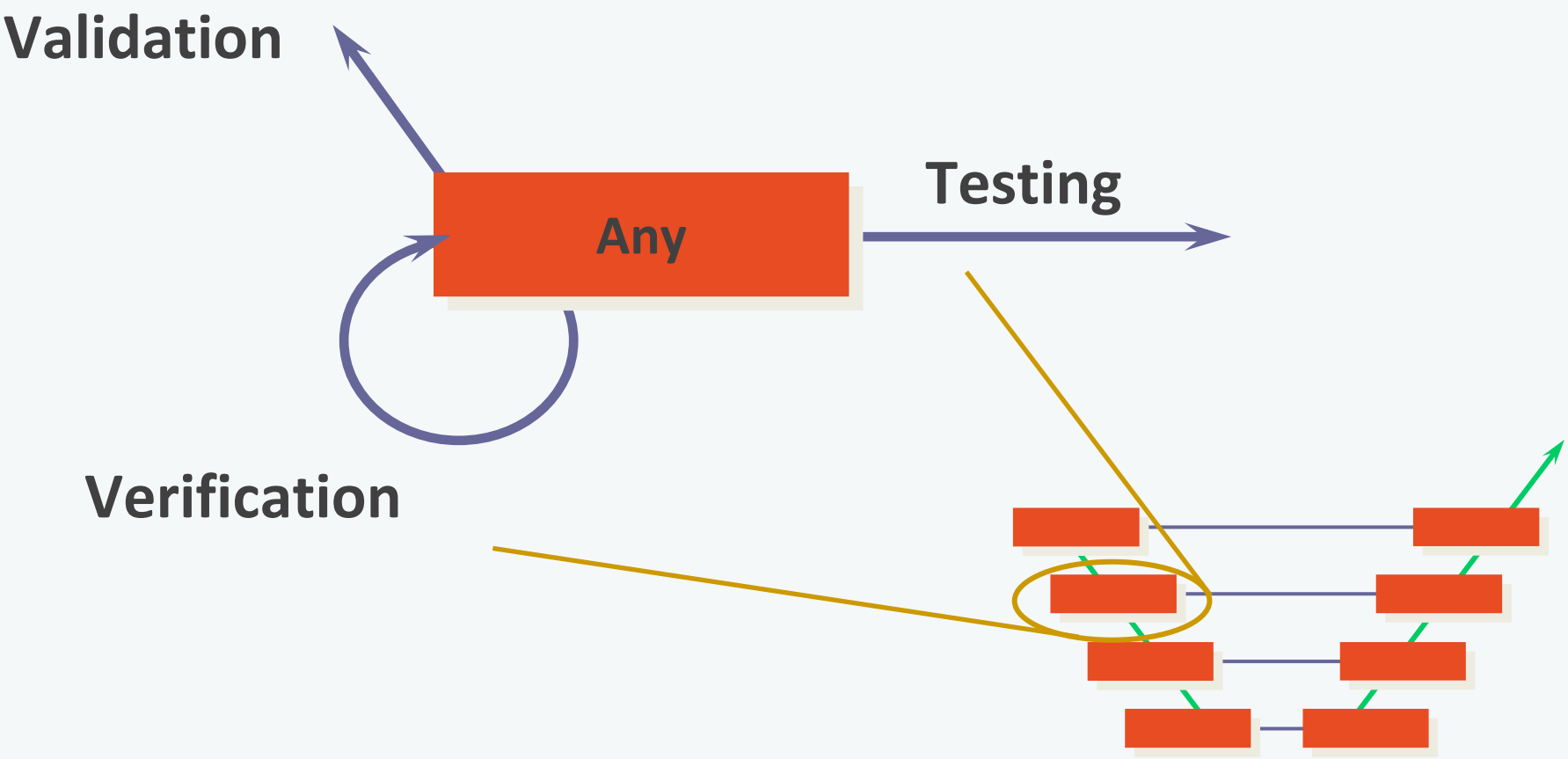
- Test design finds faults
- Faults found early are cheaper to fix
- Most significant faults found first
- Faults prevented, not built in
- No additional effort, re-schedule test design
- Changing requirements caused by test design

Early test design helps to build quality, stops fault multiplication.

VV&T

- Verification
  - the process of evaluating a system or component to determine whether the products of the given development phase satisfy the conditions imposed at the start of that phase [BS 7925-1]
- Validation
  - determination of the correctness of the products of software development with respect to the user needs and requirements [BS 7925-1]
- Testing
  - the process of exercising software to verify that it satisfies specified requirements and to detect faults

Verification, Validation and Testing



Incremental Development Models

- Incremental development **involves establishing requirements, designing, building, and testing a system in pieces**, which means that the software's features grow incrementally.
- The size of these feature increments vary, with some methods having larger pieces and some smaller pieces.
  - The feature increments can be as small as a single change to a user interface screen or new query option.
- This approach produces working versions of parts of the system early on & each of these can be released to the customer.

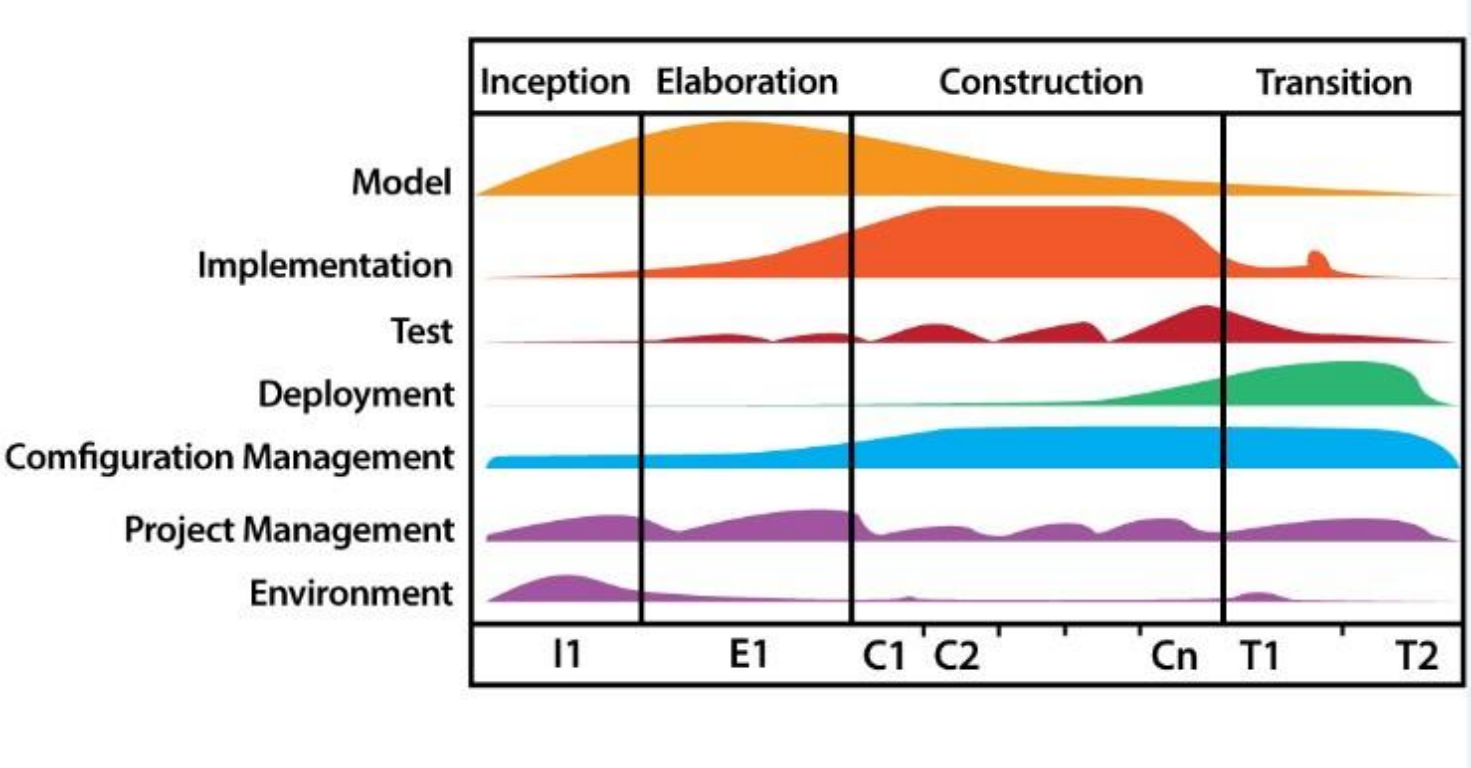
Iterative Development Models

- Start with a rough product and refine it, iteratively (rework strategy).
- Iterations may involve changes to features developed in earlier iterations, along with changes in project scope.
- Final version only delivered to customer
  - in practice, intermediate versions may be delivered to selected customers to get feedback
- Each iteration delivers working software which is a growing subset of the overall set of features until the final software is delivered or development is stopped.

Testing in Incremental & Iterative Development

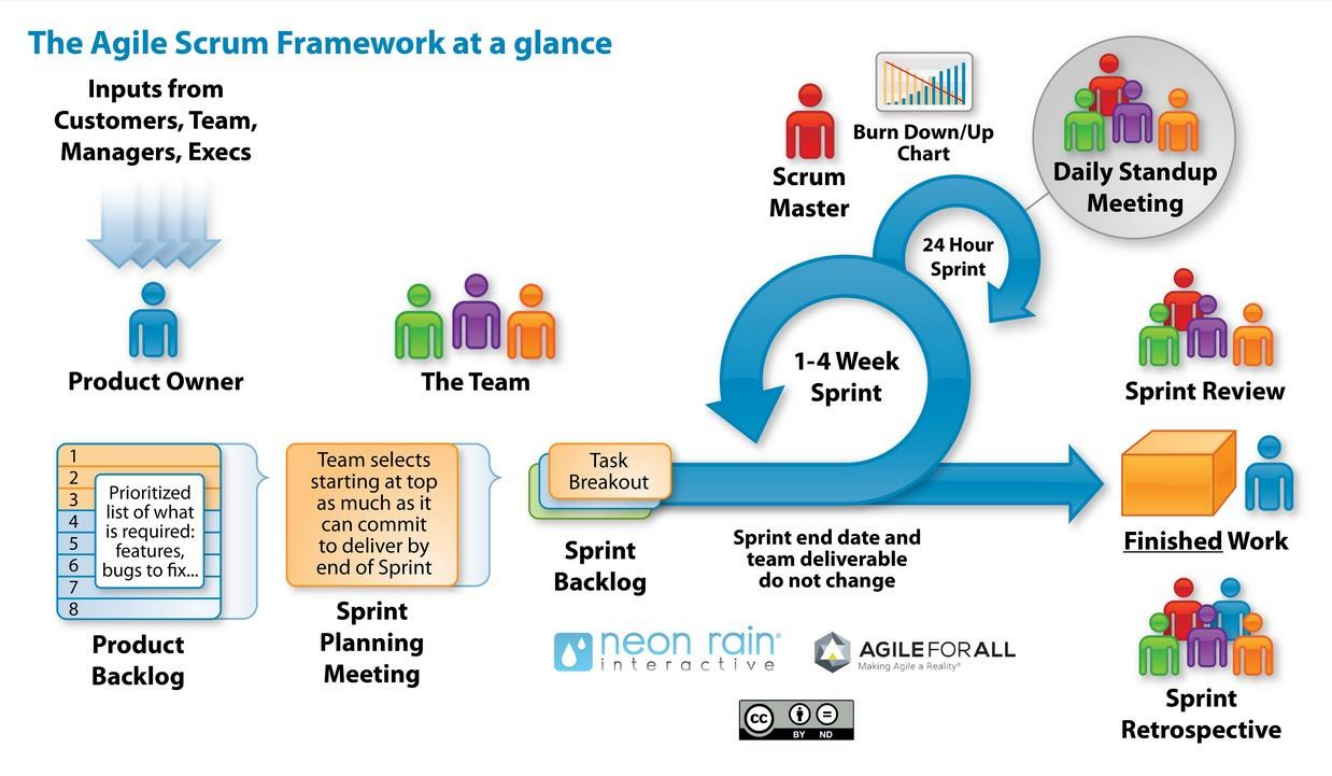
- High-level test planning & test analysis occurs at the onset of the project. Detailed test planning, analysis, design, and implementation occurs at the start of each iteration/increment.
- Test execution involves overlapping test levels.
- Many of the same tasks are performed but with varied timing and extent.
- Common issues
  - More regression testing
  - Defects outside the scope of the iteration/increment
  - Less thorough testing

Rational Unified Process (RUP)



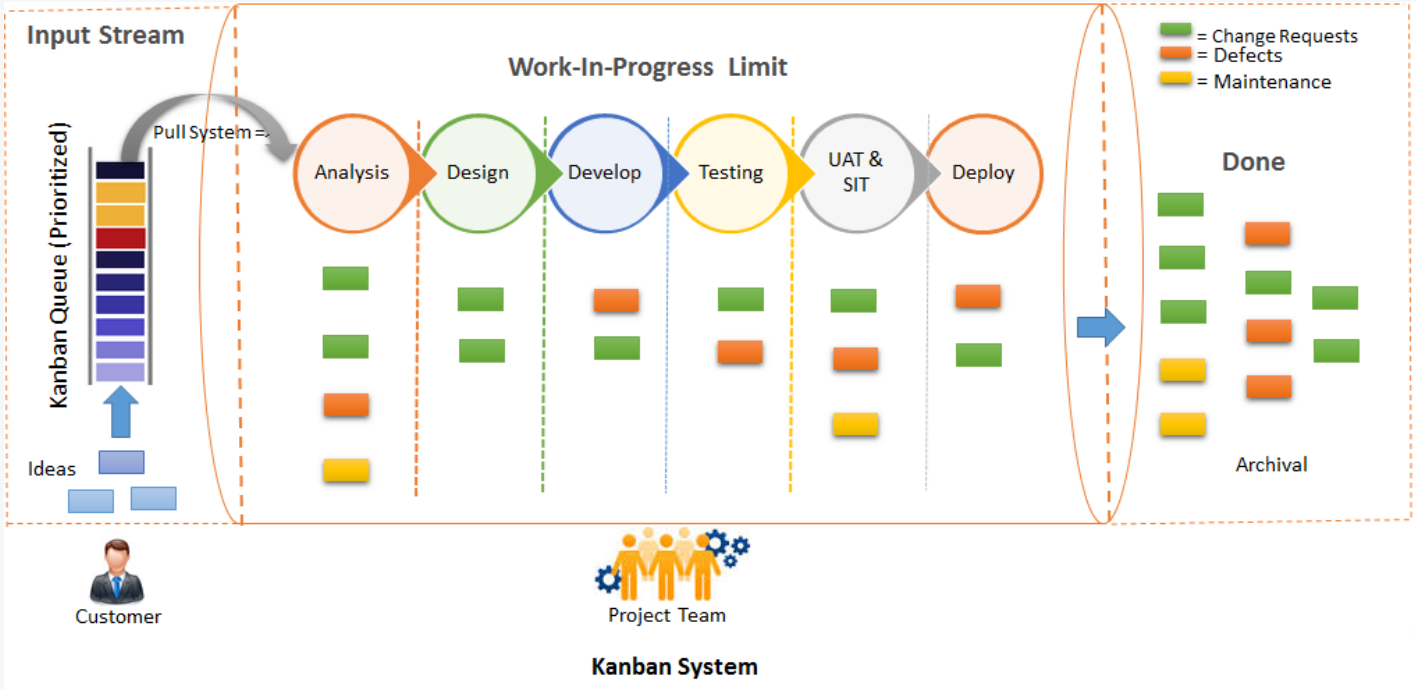
- Development is iterative with risk being the primary driver for decisions. Evaluation of quality (incl. testing) is continuous throughout development.
- Iterations tends to be relatively long (months), and feature increments are correspondingly large (e.g., 2 or 3 groups of related features).

Scrum



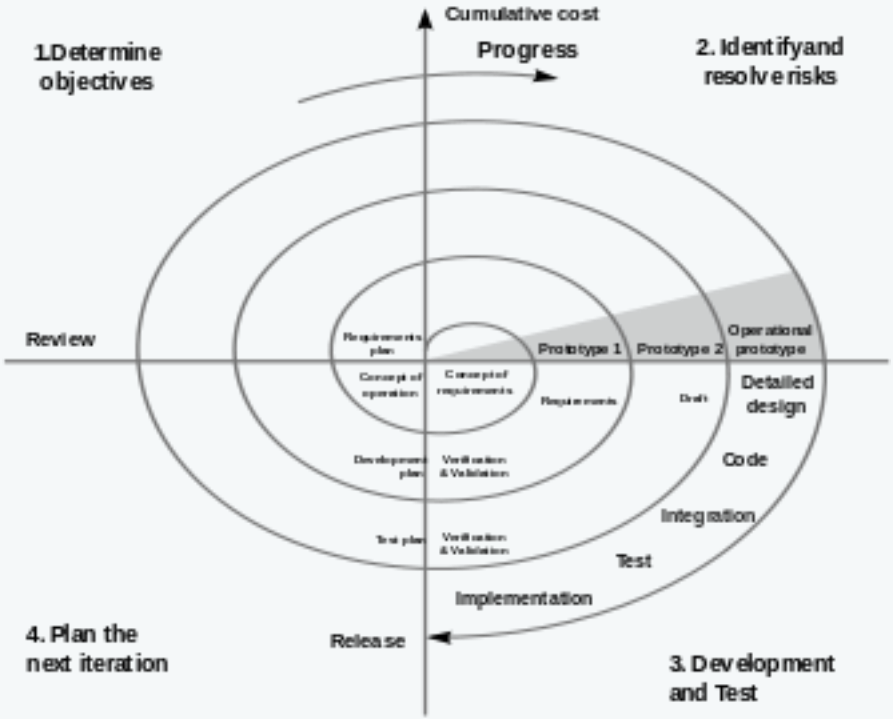
Each iteration tends to be relatively short (e.g., days, or a few weeks). Feature increments are correspondingly small (a few enhancements and/or two or three new features).

Kanban



- Implemented with or without fixed-length iterations, which can deliver either a single enhancement or feature upon completion, or can group features together to release at once.
- Key principle: to have a limit for work-in-progress (WIP) activities.

Spiral (or Prototyping)



Involves creating experimental increments, some of which may be heavily re-worked or even abandoned in subsequent development work.

Agile development

- Generation of business stories to define the functionality.
- On-site customer for continual feedback and to define & perform functional acceptance testing.
- Pair programming and shared code ownership amongst the developers.
- Component test scripts shall be written before the code is written (TDD) and that those tests should be automated.
- Simplicity: building only what is necessary, not everything we can think of.
- Continuous integration & testing of the code throughout the sprint, at least once a day.

Agile development: Benefits for Testers

- Focus on working software & good quality code
- Inclusion of testing as part of & starting point of SWD
- Accessibility of business stakeholders → Qs on systems resolved
- Self-organising team → more autonomy for testers
- Design simplicity → easier to test

Agile development: Challenges for Testers

- Different kind of test basis – less formal & subject to change
- Misperception that testers are not needed
- Different roles of tester – more like coaches
- (Usual) constant time pressure
- Risk of inadequate automated regression suite

CONTENT

- Software Development Life Cycle Models
- Test levels
- Test types
- Maintenance testing

(Before planning for a set of tests)

- Set organisational test strategy
- Identify people to be involved (sponsors, testers, QA, development, support, etc.)
- Examine the requirements or functional specifications (test basis)
- Set up the test organisation and infrastructure
- Defining test deliverables & reporting structure

See: Structured Testing, an introduction to TMap®, Pol & van Veenendaal, 1998

High level test planning

- What is the purpose of a high level test plan?
  - Who does it communicate to? – all parties involved
  - Why is it a good idea to have one?
- What information should be in a high level test plan?
  - What is your standard for contents of a test plan?
  - Have you ever forgotten something important?
  - What is not included in a test plan?

High-level Test Plan

1. Test Plan Identifier

2. Introduction
   - Software items and features to be tested
   - References to project authorisation, project plan, QA plan, CM plan, relevant policies & standards

3. Test items
   - Test items including version/revision level
   - How transmitted (net, disc, CD, etc.)
   - References to software documentation

Source: ANSI/IEEE Std 829-1998, Test Documentation

High-level Test Plan (cont.)

4. Features to be tested
   - Identify test design specification / techniques
5. Features not to be tested
   - Reasons for exclusion

High-level Test Plan (cont.)

6. Approach
   - activities, techniques and tools
   - detailed enough to estimate (*cost?)*
   - specify degree of comprehensiveness (e.g. coverage) and other completion criteria (e.g. faults)
   - identify constraints (environment, staff, deadlines)

7. Item Pass/Fail Criteria

8. Suspension criteria and resumption criteria
   - for all or parts of testing activities
   - which activities must be repeated on resumption

High-level Test Plan (cont.)

9. Test Deliverables

- Test plan
- Test design specification
- Test case specification
- Test procedure specification
- Test item transmittal reports
- Test logs
- Test incident reports
- Test summary reports

High-level Test Plan (cont.)

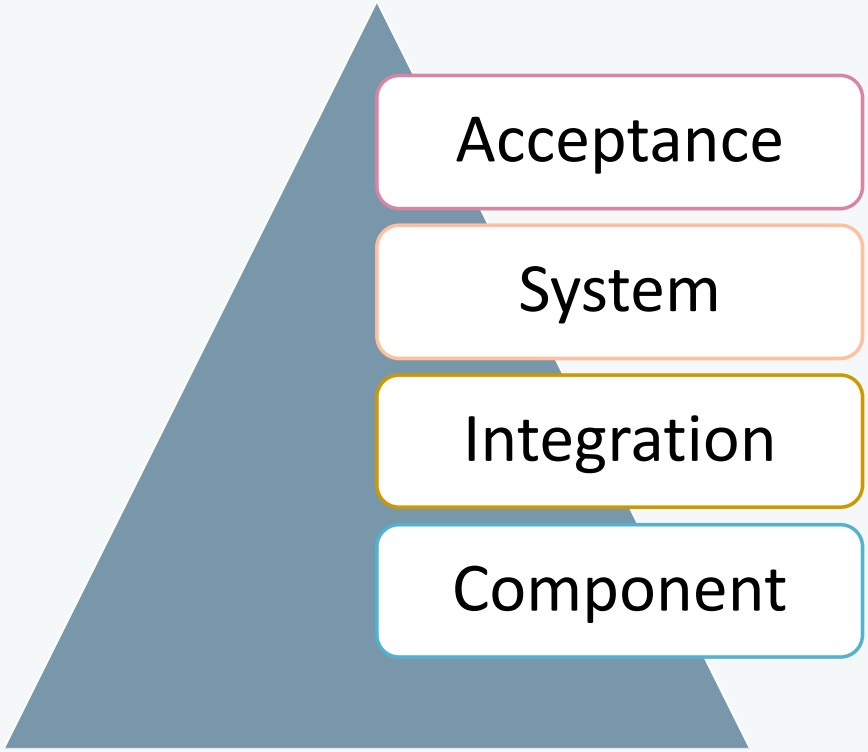
10. Testing tasks
    - including inter-task dependencies & special skills
11. Environment
    - physical, hardware, software, tools
    - mode of usage, security, office space
12. Responsibilities
    - to manage, design, prepare, execute, witness, check, resolve issues, providing environment, providing the software to test

High-level Test Plan (cont.)

13. Staffing and Training Needs
14. Schedule
    - test milestones in project schedule
    - item transmittal milestones
    - additional test milestones (environment ready)
    - what resources are needed & when
15. Risks and Contingencies
    - contingency plan for each identified risk
16. Approvals
    - names and when approved

Test Levels

- Test levels are groups of test activities that are organized and managed together.
- Each test level (test stage) is a specific instantiation of a test process.
- Test levels are related to other activities within the software development lifecycle.



Test Levels: Characteristics

Test levels are characterized by the following attributes:

- Specific test objectives
- Test basis, referenced to derive test cases
- Test object (i.e., what is being tested)
- Typical defects and failures
- Specific approaches and responsibilities

Test Levels: Environment

For every test level, a suitable test environment is required.

- In component testing, developers often use their dev environment.
- In system testing, an environment may be needed with particular external connection.
- In acceptance testing, a production-like test environment is ideal.

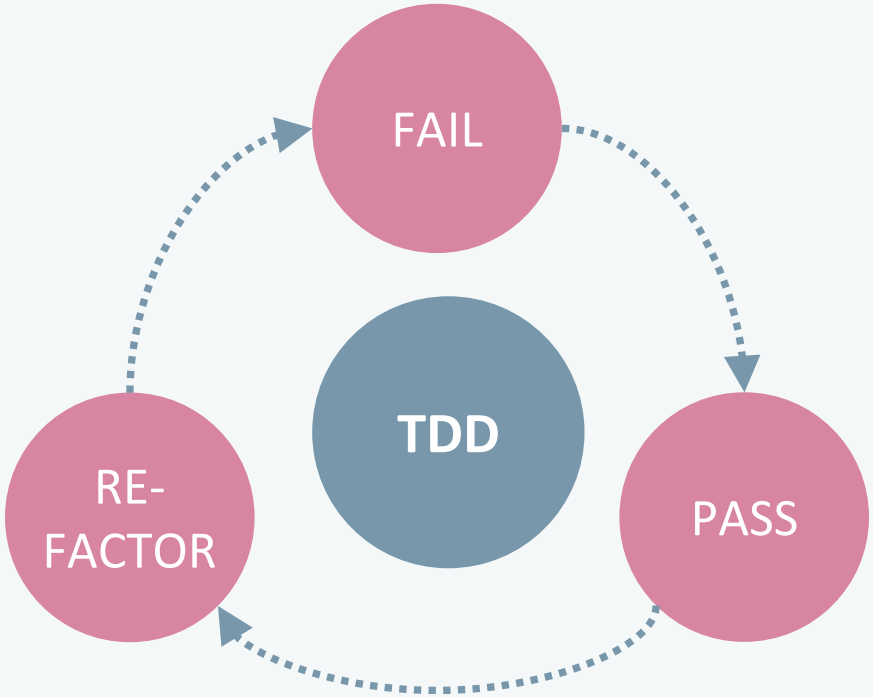
Component Testing

- Lowest level
- Tested in isolation – use of stubs and/or drivers
- Most thorough look at detail
  - Error handling
  - Interfaces
- Also known as unit, module, program testing

Component Testing

Objectives	Reduce risk. Verify functional & non-functional behaviours. Build confidence. Find defects. Prevent defects.
Test Basis	Detailed design. Code. Data model. Component specifications.
Test Objects	Component, unit, modules. Code & data structure. Classes. Database models.
Typical Defects & Failures	Incorrect functionality. Data flow problems. Incorrect code or logic.
Approaches & Responsibilities	Test-driven development (TDD). Usually done by developer

Component Testing: Test Driven Development



Developing automated test cases → building and integrating small pieces of code → executing the component tests, correcting any issues, and re-factoring the code.

Component test strategy 1

- specify test design techniques and rationale
  - from Section 3 of the standard*
- specify criteria for test completion and rationale
  - from Section 4 of the standard
- document the degree of independence for test design
  - component author, another person, from different section, from different organisation, non-human

*Source: BS 7925-2, Software Component Testing Standard

Component test strategy 2

- component integration and environment
  - isolation, top-down, bottom-up, or mixture
  - hardware and software
- document test process and activities
  - including inputs and outputs of each activity
- affected activities are repeated after any fault fixes or changes
- project component test plan
  - dependencies between component tests

Test design techniques

Also a measurement technique?

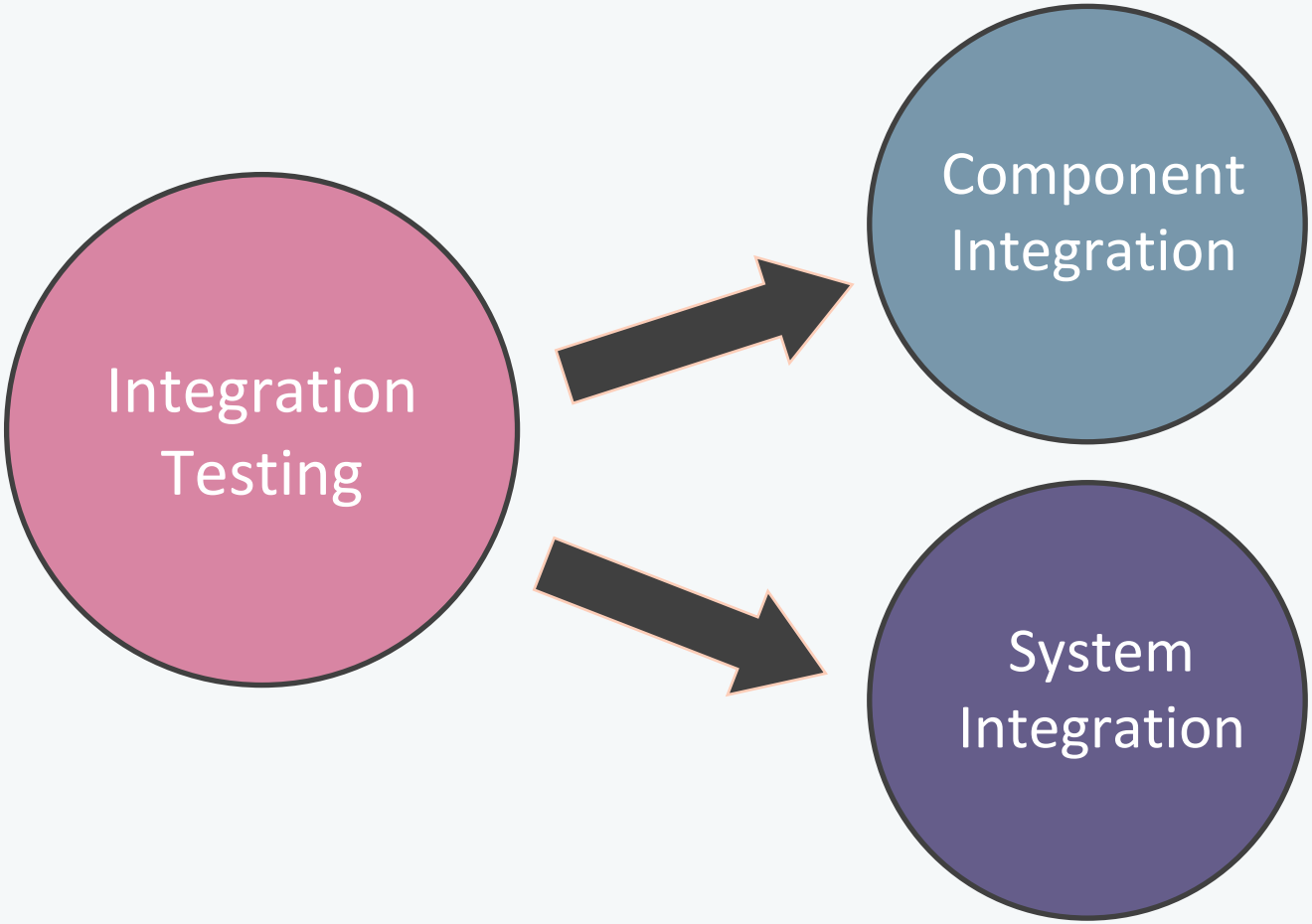
✓ **= Yes**

✗ **= No**

- "Black box"
  - Equivalence partitioning ✓
  - Boundary value analysis ✓
  - State transition testing ✓
  - Cause-effect graphing ✓
  - Syntax testing ✗
  - Random testing ✗
- How to specify other techniques
- "White box"
  - Statement testing ✓
  - Branch / Decision testing ✓
  - Data flow testing ✓
  - Branch condition testing ✓
  - Branch condition combination testing ✓
  - Modified condition decision testing ✓
  - LCSAJ testing ✓

Integration Testing

Integration testing focuses on interactions between components or systems.



Integration Testing

- Component integration tests and system integration tests should concentrate on the **integration** itself.
- If integrating module A with module B, tests should focus on the **communication between the modules**, **<u>not</u> the functionality of the individual modules**, as that should have been covered during component testing.
- If integrating system X with system Y, tests should focus on the **communication between the systems**, **<u>not</u> the functionality of the individual systems**, as that should have been covered during system testing.

Integration Testing

- Component integration testing is often the responsibility of developers.
- System integration testing is generally the responsibility of testers.
- To simplify defect isolation and detect defects early, integration should normally be incremental.
- The greater the scope of integration, the more difficult it becomes to isolate defects to a specific component/system → continuous integration (i.e., software is integrated on a component-by-component basis)

Integration Testing

Objectives	Reduce risk. Verify functional & non-functional behaviours of interfaces. Build confidence. Find defects. Prevent defects.
Test Basis	Software & system design. Sequence diagrams. Interface & communication protocol specs. Use cases. Workflows.
Test Objects	Subsystems. Databases. Infrastructure. Interfaces. APIs. Microservices.
Typical Defects & Failures	Incorrect data. Incorrect timing. Interface mismatch. Communication failures b/w components. Incorrect assumptions
Approaches & Responsibilities	Big-bang. Incremental (top-down, bottom-up, functional)

Big-Bang Integration

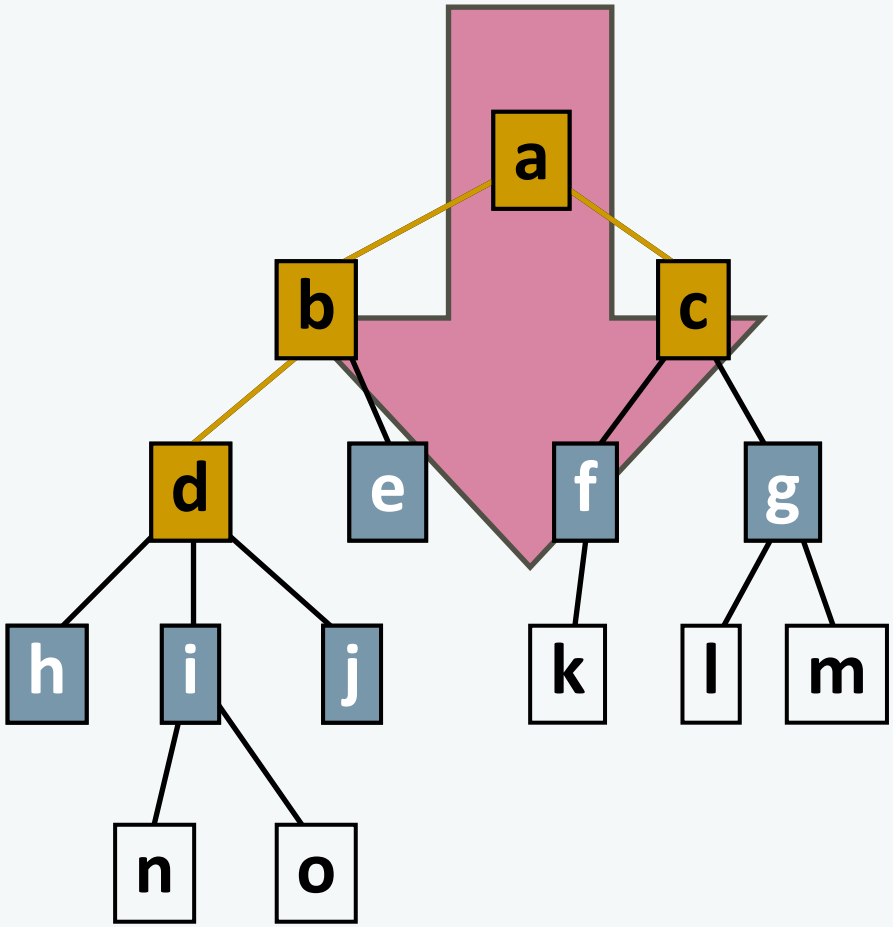
- In theory:
  - if we have already tested components why not just combine them all at once? Wouldn't this save time?
  - (based on false assumption of no faults)
- In practice:
  - takes longer to locate and fix faults
  - re-testing after fixes more extensive
  - end result? takes more time

Incremental Integration

- Baseline 0: tested component
- Baseline 1: two components
- Baseline 2: three components, etc.
- Advantages:
  - easier fault location and fix
  - easier recovery from disaster / problems
  - interfaces should have been tested in component tests, but ..
  - add to tested baseline

Top-Down Integration

- Baselines:
  - baseline 0: component a
  - baseline 1: a + b
  - baseline 2: a + b + c
  - baseline 3: a + b + c + d
  - etc.
- Need to call to lower level components not yet integrated
- Stubs: simulate missing components



Stubs

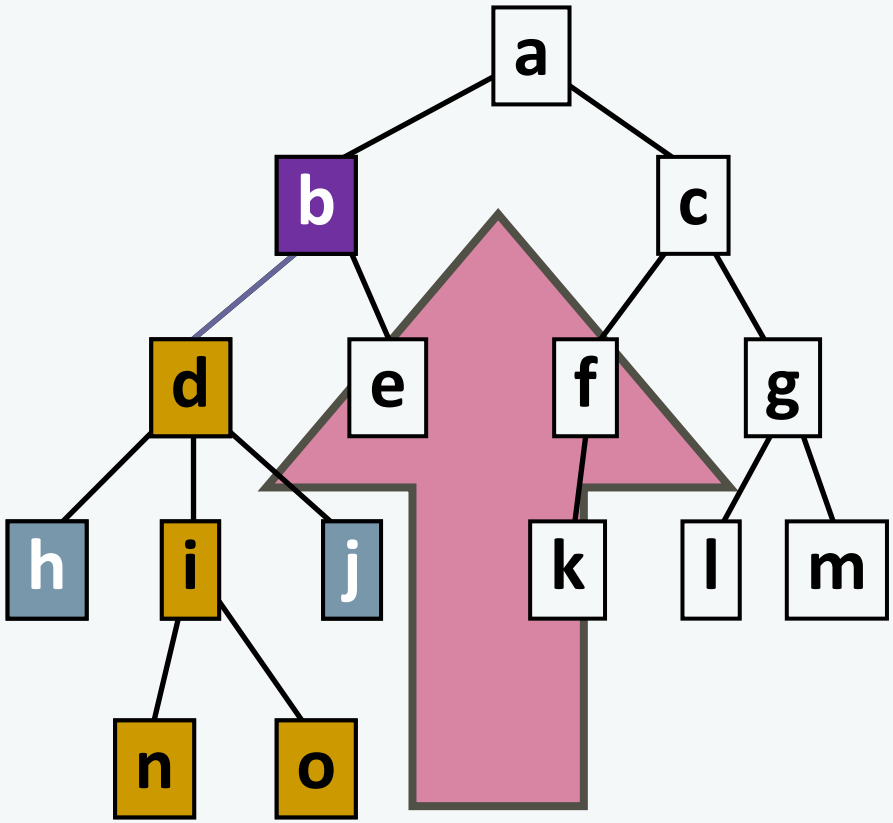
- Stub (Baan: dummy sessions) replaces a called component for integration testing
- Keep it Simple
  - print/display name (I have been called)
  - reply to calling module (single value)
  - computed reply (variety of values)
  - prompt for reply from tester
  - search list of replies
  - provide timing delay

Pros & cons of top-down approach

- Advantages:
  - Critical control structure tested first and most often
  - Can demonstrate system early (show working menus)
- Disadvantages:
  - Needs stubs
  - Detail left until last
  - May be difficult to "see" detailed output (but should have been tested in component test)
  - May look more finished than it is

Bottom-up Integration

- Baselines:
  - baseline 0: component n
  - baseline 1: n + i
  - baseline 2: n + i + o
  - baseline 3: n + i + o + d
  - etc.
- Needs drivers to call the baseline configuration
- Also needs stubs for some baselines



Drivers

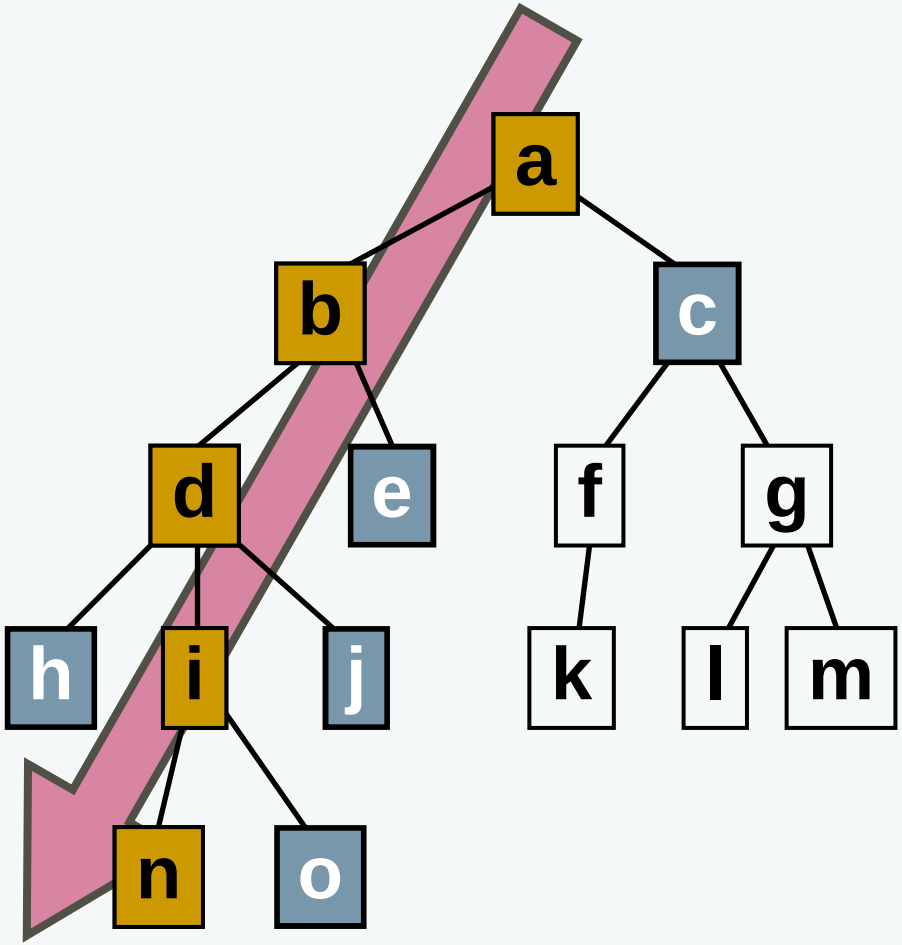
- Driver (Baan: dummy sessions): test harness: scaffolding
- Specially written or general purpose (commercial tools)
  - invoke baseline
  - send any data baseline expects
  - receive any data baseline produces (print)
- Each baseline has different requirements from the test driving software.

Pros & cons of bottom-up approach

- Advantages:
  - lowest levels tested first and most thoroughly (but should have been tested in unit testing)
  - good for testing interfaces to external environment (hardware, network)
  - visibility of detail
- Disadvantages
  - no working system until last baseline
  - needs both drivers and stubs
  - major control problems found last

Minimum Capability Integration (aka. Functional)

- Baselines:
  - baseline 0: component a
  - baseline 1: a + b
  - baseline 2: a + b + d
  - baseline 3: a + b + d + i
  - etc.
- Needs stubs
- Shouldn't need drivers (if top-down)

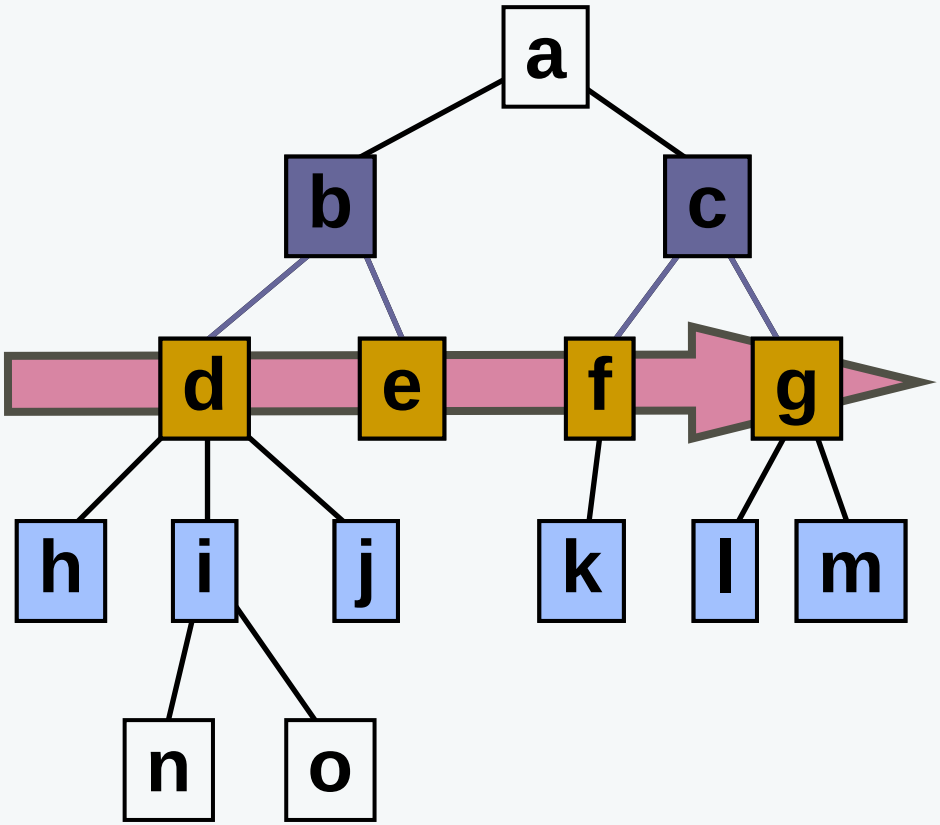


Pros & cons of Minimum Capability

- Advantages:
  - Control level tested first and most often
  - Visibility of detail
  - Real working partial system earliest
- Disadvantages
  - Needs stubs

Thread Integration (also called functional)

- Order of processing some event determines integration order
- Interrupt, user transaction
- Minimum capability in time
- Advantages:
  - Critical processing first
  - Early warning of performance problems
- Disadvantages:
  - may need complex drivers and stubs



Integration Guidelines

- Minimise support software needed
- Integrate each component only once
- Each baseline should produce an easily verifiable result
- Integrate small numbers of components at once
  - one at a time for critical or fault-prone components
  - combine simple related components

Integration Planning

- Integration should be planned in the architectural design phase
- The integration order then determines the build order
  - Components completed in time for their baseline
  - Component development and integration testing can be done in parallel - saves time

System Testing

System testing focuses on the behaviour and capabilities of a whole system or product, often considering the end-to-end tasks the system can perform and the non-functional behaviours it exhibits while performing those tasks.

System Testing

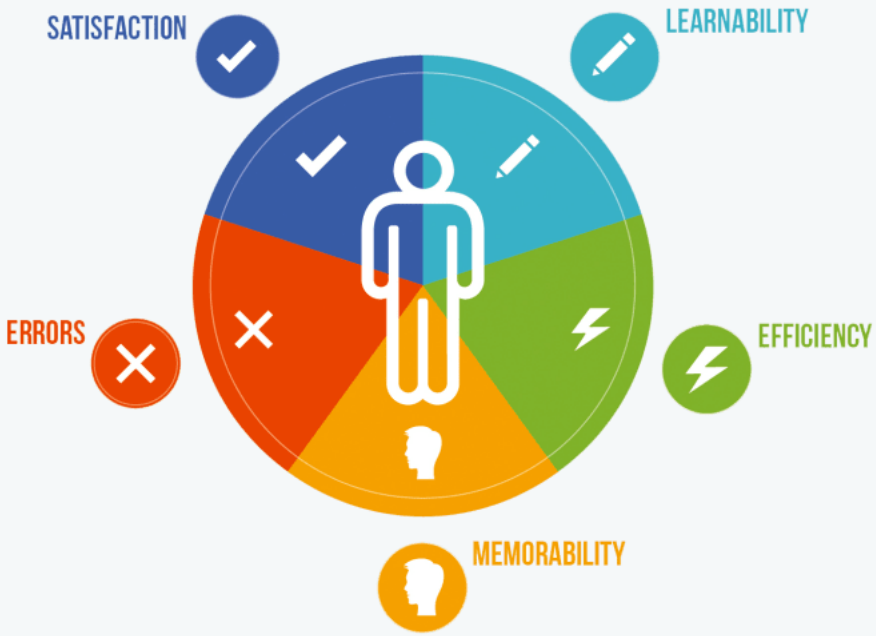
Objectives	Reduce risk. Verify functional & non-functional behaviours of system. Validate system is complete & as expected. Build confidence. Find & Prevent defects.
Test Basis	Software & system reqs specs. Risk analysis reports. Use cases. Epics & user stories. System models. State diagrams. System & User manuals.
Test Objects	Applications. Hardware/software. Operating system. SUT. System configuration & config data.
Typical Defects & Failures	Incorrect calculations. Incorrect/unexpected system (non-)functional behaviours. Incorrect data flows. Cannot complete end-to-end tasks. Not as described in manuals.

System Testing: Approaches & Responsibilities

- Independent testers typically carry out system testing.
- System testing of functional reqs starts by using most appropriate black-box techniques (e.g., decision table). White-box techniques may be used to assess the thoroughness of testing elements (e.g., menu dialogue structure, web page navigation).
- The (properly controlled) test environment should ideally correspond to the final target or production environment.

Acceptance Testing

Acceptance testing. Formal testing with respect to user needs, requirements, and business processes conducted to determine whether or not a system satisfies the acceptance criteria and to enable the user, customers or other authorised entity to determine whether or not to accept the system. (Textbook, p.55)

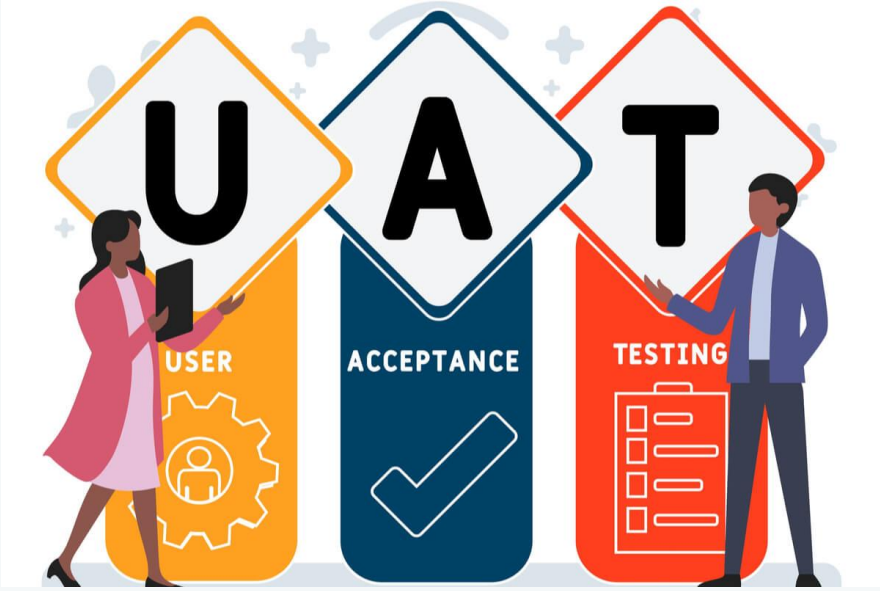


Acceptance Testing

- Acceptance testing may produce information to assess the system's readiness for deployment and use by the customer (end-user).
- Defects may be found during acceptance testing, but finding defects is often **not** an objective, and finding a significant number of defects during acceptance testing may in some cases be considered a major project risk.

Acceptance Testing: UAT

- Done by end-users
- Focus: business processes
- Environment: real / simulated operational environment
- Aim: to build confidence that system will enable users to perform what they need to do with a minimum of difficulty, cost, and risk



User acceptance testing

- Final stage of validation
  - Customer (user) should perform or be closely involved
  - Customer can perform any test they wish, usually based on their business processes
  - Final user sign-off
- Approach
  - Mixture of scripted and unscripted testing
  - "Model Office" concept sometimes used

Why customer / user involvement

- Users know:
  - what really happens in business situations
  - complexity of business relationships
  - how users would do their work using the system
  - variants to standard tasks (e.g. country-specific)
  - examples of real cases
  - how to identify sensible work-arounds

Benefit: detailed understanding of the new system

Acceptance Testing: OAT

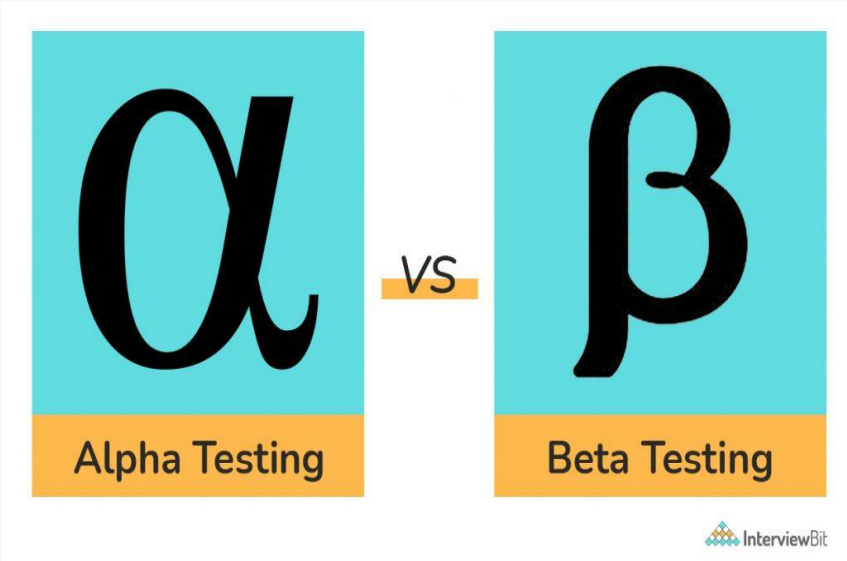
- Done by system admins
- Focus: backups; installation, uninstallation upgrading; disaster recovery; user management; maintenance; data loading & migrations; security; performance
- Environment: simulated production environment
- Aim: to give confidence to the system admins that they will be able to keep the system running & recover from adverse events quickly and w/o additional risks.

Acceptance Testing: C/RAT

- Contractual AT: to verify whether a system satisfies its contractual requirements. Performed by users / independent testers.
- Regulatory AT: to verify whether a system conforms to relevant laws, policies and regulations. Performed by independent testers (possibly with a representative of regulatory body.

Acceptance Testing: Alpha & Beta Testing

- Alpha testing. Simulated or actual operational testing conducted in the developer's test environment, by roles outside the development organization.
- Beta testing (field testing). Simulated or actual operational testing conducted at an external site, by roles outside the development organisation → diverse users; various environments → testing can cover more combinations of factors.



Acceptance Testing

Objectives	Establish confidence. Validate the system is complete & as expected. Verify functional & non-functional behaviours as specified.
Test Basis	Biz process. User/Biz reqs. Regulations, legal contract & standards. Use cases. System reqs. System/User documentation. Risk analysis reports. Backup & recovery procedures. Disaster recovery plan. Non-functional reqs. Operations doc. Performance targets. DB packages. Security standards.
Test Objects	SUT. System configuration & config data. Recovery system. Hot sits. Forms. Reports.
Typical Defects & Failures	System workflow. Business rules. Contract. Non-functional failures (security vulnerabilities, performance inefficiency, etc)

Acceptance testing motto

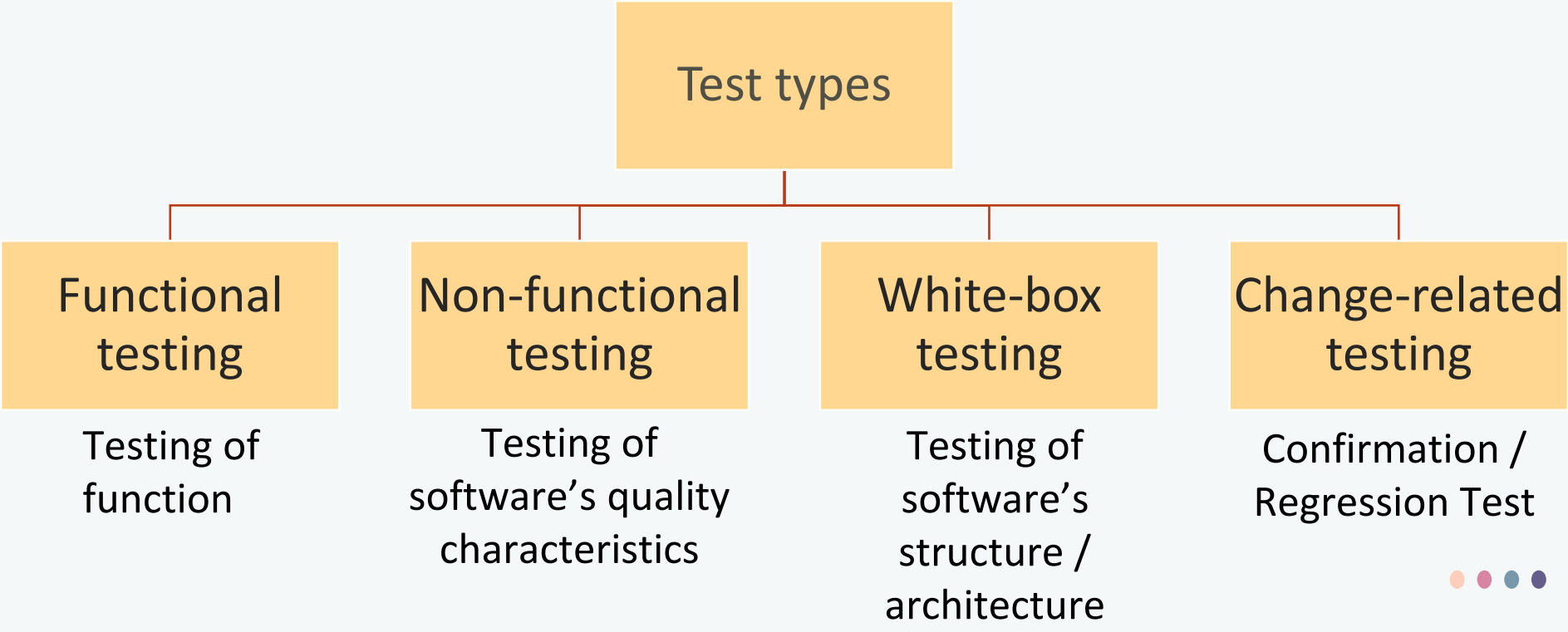
If you don't have patience to test the system, the system will surely test your patience.

CONTENT

- Software Development Life Cycle Models
- Test levels
- Test types
- Maintenance testing

Test Types

- A test type is a group of test activities aimed at testing specific characteristics of a software system, or a part of a system, based on specific test objectives.



[1] Functional Testing

- The function of a system/component is "what" it does. Testing conducted to evaluate the compliance of a component/system with functional requirements.
- Functional requirements may be described in work products such as:
  - Business reqs specs
  - Epics
  - User stories
  - Use cases
  - Functional specs
  - They may be undocumented.
- Functional tests should be performed at all test levels, though the focus is different at each level
- Can be done from 2 perspectives: requirement-based and business-process-based.

[1] Functional Testing

- Functional requirements
  - a requirement that specifies a function that a system or system component must perform (ANSI/IEEE Std 729-1983, Software Engineering Terminology)
- Functional specification
  - the document that describes in detail the characteristics of the product with regard to its intended capability (BS 4778 Part 2, BS 7925-1)

[1] Functional Testing: Requirements-based

- Uses specification of requirements as the basis for identifying tests
  - Table of contents of the requirements spec provides an initial test inventory of test conditions
  - For each section / paragraph / topic / functional area,
    - risk analysis to identify most important / critical
    - decide how deeply to test each functional area

[1] Functional Testing: Business-process-based

- Expected user profiles
  - what will be used most often?
  - what is critical to the business?
- Business scenarios
  - typical business transactions (start to finish)
- Use cases
  - prepared cases based on real situations

[1] Functional Testing: Coverage

- Functional coverage is the extent to which some type of functional element has been exercised by tests, and is expressed as a percentage of the type(s) of element being covered.
- Using traceability between tests and functional requirements, the percentage of these requirements which are addressed by testing can be calculated, potentially identifying coverage gaps.

[2] Non-functional Testing

- Non-functional testing is the testing of "how well" the system behaves
- Non-functional testing of a system evaluates characteristics of systems and software such as usability, performance, efficiency or security, etc.
- Non-functional testing can be done at all test levels.
- Defines expected results in terms of external behaviour → typically use black-box test techniques
  - BVA – stress conditions – performance testing
  - EP – types of devices – compatibility testing, or user groups – usability testing (novice, experienced, age range, geographical location, educational background)

[2] Non-functional Testing: Coverage

- The thoroughness of non-functional testing can be measured by the coverage of non-functional elements.
  - If we had at least 1 test for each major group of users, we would have 100% coverage of those user groups identified.
- Traceability between non-functional tests and non-functional requirements, we can identify coverage gaps
  - E.g., an implicit requirement is for accessibility for disabled users

Performance Tests

- Timing Tests
  - Response and service times
  - Database back-up times
- Capacity & Volume Tests
  - Maximum amount or processing rate
  - Number of records on the system
  - Graceful degradation
- Endurance Tests (24-hr operation?)
  - Robustness of the system
  - Memory allocation

Multi-User Tests

- Concurrency Tests
  - Small numbers, large benefits
  - Detect record locking problems
- Load Tests
  - The measurement of system behaviour under realistic multi-user load
- Stress Tests
  - Go beyond limits for the system - know what will happen
  - Particular relevance for e-commerce

Source: Sue Atkins, Magic Performance Management

Usability Tests

- Messages tailored and meaningful to (real) users?
- Coherent and consistent interface?
- Sufficient redundancy of critical information?
- Within the "human envelope"? (7±2 choices)
- Feedback (wait messages)?
- Clear mappings (how to escape)?

Who should design / perform these tests?

Security Tests

- Passwords
- Encryption
- Hardware permission devices
- Levels of access to information
- Authorisation
- Covert channels
- Physical security

Configuration and Installation

- Configuration Tests
  - Different hardware or software environment
  - Configuration of the system itself
  - Upgrade paths - may conflict
- Installation Tests
  - Distribution (CD, network, etc.) and timings
  - Physical aspects: electromagnetic fields, heat, humidity, motion, chemicals, power supplies
  - Uninstall (removing installation)

Reliability / Qualities

- Reliability
  - "System will be reliable" - how to test this?
  - "2 failures per year over ten years"
  - Mean Time Between Failures (MTBF)
  - Reliability growth models
- Other Qualities
  - Maintainability, Portability, Adaptability, etc.

Back-up and Recovery

- Back-ups
  - Computer functions
  - Manual procedures (where are tapes stored)
- Recovery
  - Real test of back-up
  - Manual procedures unfamiliar
  - Should be regularly rehearsed
  - Documentation should be detailed, clear and thorough

Documentation Testing

- Documentation review
  - check for accuracy against other documents
  - gain consensus about content
  - documentation exists, in right format
- Documentation tests
  - is it usable? does it work?
  - user manual
  - maintenance documentation

[3] White-box Testing

- White-box testing derives tests based on the system's internal structure or implementation of the component or system.
- Internal structure may include code, architecture, work flows, and/or data flows within the system.
- Can occurs at any test level; **but**
  - tends to mostly at component testing and component integration testing
  - Generally less likely at higher test levels, except for business process testing (test basis could be business rules)

[3] White-box Testing: Coverage

- Structural coverage is the extent to which some type of structural element has been exercised by tests, expressed as a percentage of the type of element being covered.
- At the component testing level, code coverage is based on the percentage of executable elements (e.g., statements or decision outcomes)
- At the component integration testing level, white-box testing may be based on the architecture of the system (e.g., interface between components), and coverage may be measured by percentage of interfaces exercised by tests.

[4] Change-related Testing

- When changes are made to a system, testing should be done to confirm that the changes have corrected the defect or implemented the functionality correctly, and have not caused any unforeseen adverse consequences.
- Two sub-types: Confirmation testing and Regression testing

[4] Change-related Testing: Confirmation Testing

- After a defect is fixed, the software should be re-tested.
- At the very least, the steps to reproduce the failure(s) caused by the defect must be re-executed on the new software version.
- The purpose of a confirmation test is to confirm whether the original defect has been successfully fixed.

[4] Change-related Testing: Regression Testing

- It is possible that a change made in one part of the code, may accidentally affect the behaviour of other parts of the code
- Changes may include changes to the environment
- Regression testing involves running tests to detect such unintended side-effects.

[4] Change-related Testing: Regression Testing

- Regression test suites are run many times and generally evolve slowly, so regression testing is a strong candidate for automation.
- Automation of these tests should start early in the project.
- Change-related testing is performed at all test levels.

Test Types & Test Levels

	Functional Test	Non-functional Test
Component	How components calculate compound interest	Time to perform a complex interest calculation
Component Integration	How account info from user interface is passed to the business logic	Check for buffer overflow from data passed from the UI to business logic
System	How account holders can apply for a line of credit	Portability tests of presentation layer on browsers & mobiles
System Integration	How system uses an external microservice to check an account holder's credit score	Reliability tests (robustness) if the microservice does not respond
Acceptance	How banker handles a credit application	Usability tests (accessibility) for banker's credit processing interface for the disabled

Test Types & Test Levels

	White-box Test	Change-related Test
Component	100% statement & decision coverage for all financial calculations components	Automated regression tests for each component are included in CI framework & pipeline
Component Integration	Coverage of how each screen in the browser interface passes data to the next screen in biz logic	Confirmation tests for interface-related defects are activated as fixes are checked in
System	Coverage of web page sequence during a credit line application	All tests for a given workflow are re-executed if any screen changes
System Integration	Coverage of all possible inquiry types sent to the credit score microservice	Automated tests of interactions of system with microservice are re-executed as the service is changed
Acceptance	Coverage of all supported financial data file structures & value ranges for bank-to-bank transfers	Previously failed tests are re-executed after defects found are fixed

CONTENT

- Software Development Life Cycle Models
- Test levels
- Test types
- Maintenance testing

Maintenance testing

- Testing to preserve quality:
  - Different sequence
    - Development testing executed bottom-up
    - Maintenance testing executed top-down
    - Different test data (live profile)
  - Breadth tests to establish overall confidence
  - Depth tests to investigate changes and critical areas
  - Predominantly regression testing

What to test in maintenance testing

- Triggers for maintenance: Modification – Migration – Retirement
- Impact analysis
  - What could this change have an impact on?
  - How important is a fault in the impacted area?
  - Test what has been affected, but how much?
    - Most important affected areas?
    - Areas most likely to be affected?
    - Whole system?
- The answer: "It depends"

Poor or missing specifications

- Consider what the system should do
  - talk with users
- Document your assumptions
  - ensure other people have the opportunity to review them
- Improve the current situation
  - document what you do know and find out
- Track cost of working with poor specifications
  - to make business case for better specifications

What should the system do?

- Alternatives
  - the way the system works now must be right (except for the specific change)
  - use existing system as the baseline for regression tests
  - look in user manuals or guides (if they exist)
  - ask the experts - the current users
- Without a specification, you cannot really test, only explore. You can validate, but not verify.